

Sistem Algoritması Tasarımı

Robot Kol ve Otonom Araç

TEKNOFEST 2026 Mesleki Yetenek Yarışması
Akıllı Fabrika Sistemleri Programlama Kategorisi, Yarı Final Görev 1

Takım: andromeda

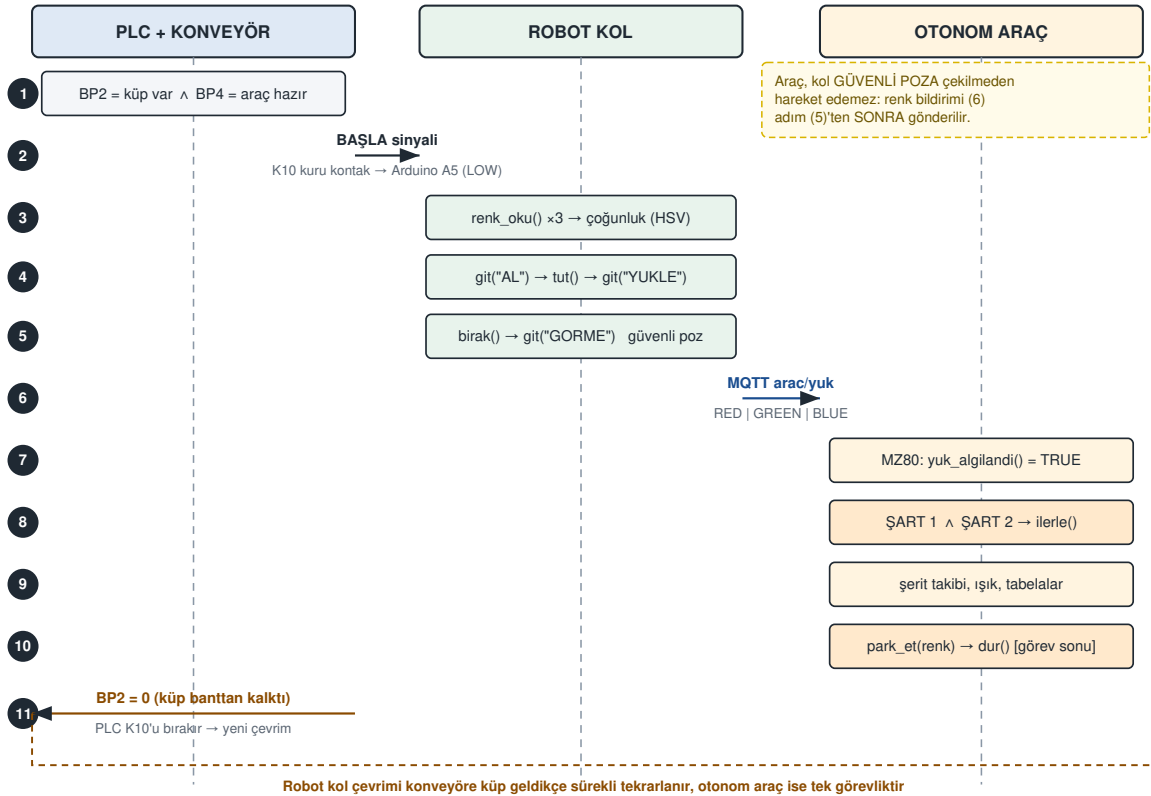
Bu doküman robot kol ile otonom aracı birlikte çalıştıran algoritmayı anlatıyor. İçinde sistemin genel işleyişi, dört akış şeması, tam sözde kod, tasarım kararlarının gerekçeleri ve dayandığımız varsayımlar var.

Tasarımda yalnızca bize verilen hazır eylemleri kullandık. Bunların dışında hiçbir donanım çağırısı, sensör erişimi ya da kütüphane fonksiyonu varsaymadık.

1. Sistemin genel işleyişi

Sistemde üç ayrı denetleyici var ve bunlar iki farklı kanaldan haberleşiyor. PLC hattı yönetiyor ve robot kola tek bir dijital başla sinyali gönderiyor. Sinyali K10 rölesinin kuru kontağı taşıyor, Arduino'nun A5 ucunu şaseye çekiyor. Robot kol küpün rengini görüntüyle işlemeye belirliyor, küpü araca yüklüyor ve rengi MQTT üzerinden arac/yuk konusuna yayınlıyor. Otonom araç da bu rengi dinliyor, küpün üzerinde olduğunu MZ80 ile doğruluyor, sonra pist görevlerini tamamlayıp renge uygun park alanına giriyor.

İki sürecin ömrü farklı ve bu ayırım tasarımın temeli. Robot kol süreci sonsuz döngü, konveyöre küp geldikçe çevrim tekrarlanır. Otonom araç süreci ise tek görevlik, park etmesiyle birlikte sona eriyor.



Şekil 1: Sistem etkileşim diyagramı

2. Kullandığımız hazır eylemler

Aşağıdaki iki tabloda bize verilen hazır eylemlerin tamamı ve her birini bu tasarımda nerede kullandığımız var. Tasarım bu kümenin dışına çıkmıyor.

Robot kol

Eylem	Ne yapar	Nerede kullandık
home()	Tüm eksenleri referans konumuna götürür.	Sadece başlangıçta bir kez. Güç açılışında kolun nerede durduğunu bilmiyoruz.
git("GORME")	Öğretilmiş poza gider.	Bekleme pozu, kamera buradan konveyör çıkışını görüyor. Yüklemeden sonra da güvenli poz olarak aynı yere dönüyoruz.
git("AL")		Küpü kavrama pozu.
git("YUKLE")		Küpü araca bırakma pozu.
tut()	Tutucuyu kapatır.	AL pozunda, küpü kavramak için.
birak()	Tutucuyu açar.	YUKLE pozunda küpü bırakırken. Ayrıca başlangıçta tutucuyu boşaltmak için.
renk_oku()	RED, GREEN, BLUE veya UNKNOWN döndürür.	GORME pozunda üç kez çağırıp çoğunluk oyu alıyoruz (Şekil 4).
başla_geldi()	PLC başla sinyali, True veya False.	Çevrim başında beklemek için, bir de çevrim sonunda sinyalin düşmesini beklemek için.
araca_gonder(renk)	Rengi MQTT ile araca bildirir.	Çevrimin en son adımı, kol güvenli poza çekildikten sonra.

Otonom araç

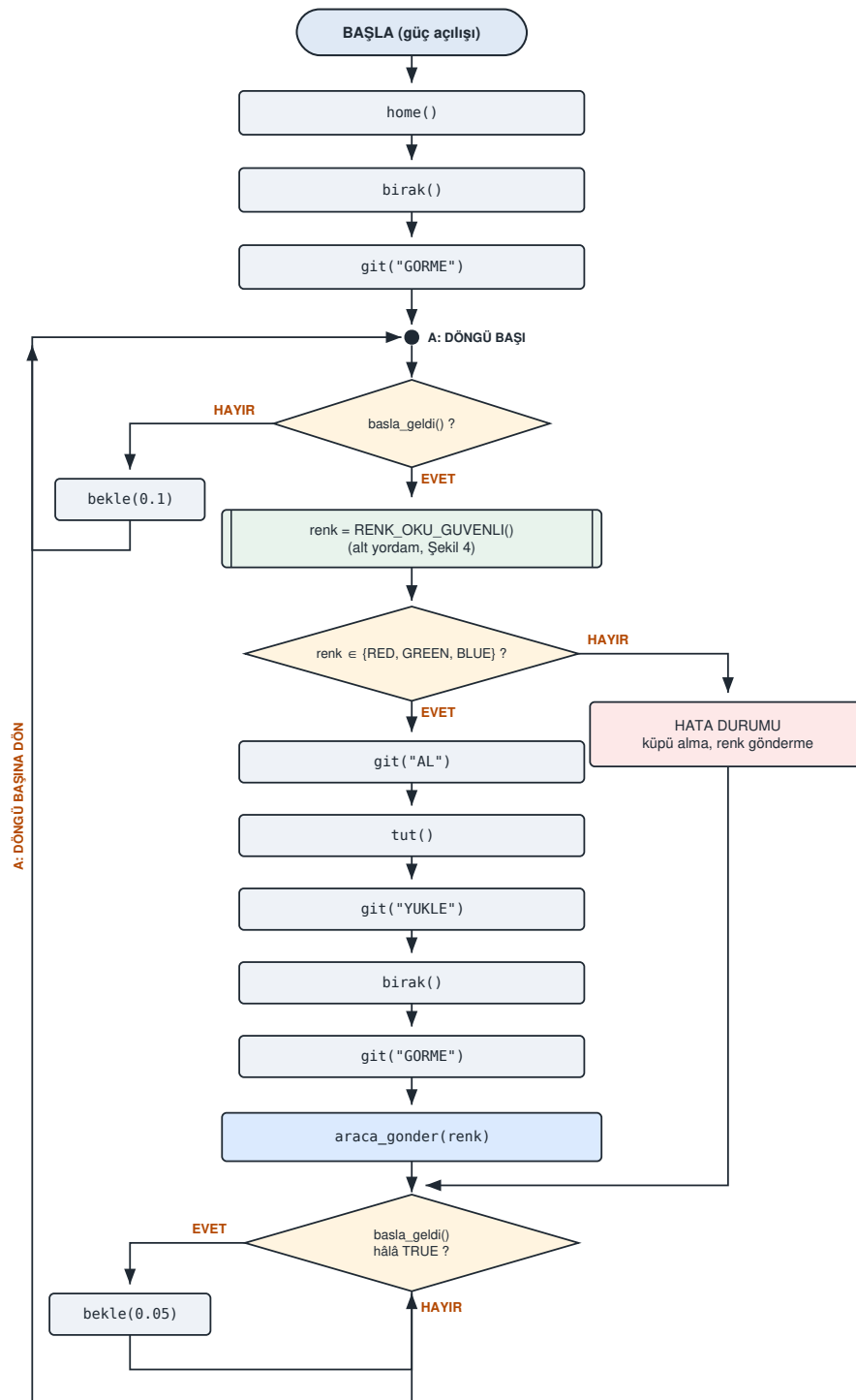
Eylem	Ne yapar	Nerede kullandık
yuk_rengi_bekle()	MQTT'den renk gelene kadar bekler.	Birinci şart. Bloklayıcı olduğu için süreç burada duruyor.
yuk_algilandi()	MZ80 küpü gördüyse True.	İkinci şart. Sağlanana kadar yokluyoruz, araç hareket etmiyor.
ilerle()	Aracı ilerletir.	Yalnızca iki şart birlikte sağlandıktan sonra, bir de her duruşun ardından.
dur()	Aracı durdurur.	Başlangıçta, kırmızı ışıktaki, yaya ve tümsek görevlerinde, park sonunda.
bekle(sn)	Verilen süre kadar bekletir.	Yoklama aralığı, tabelaya yaklaşma süresi ve görev süreleri.
isik_oku()	KIRMIZI, YESIL veya YOK döndürür.	Trafik ışığı alt yordamında (Şekil 5).
tabela_oku()	YAYA, TUMSEK, PARK veya YOK döndürür.	Görev döngüsünün her turunda, kilitleme mantığıyla (Şekil 3).

Eylem	Ne yapar	Nerede kullandık
park_et (renk)	Verilen renkteki alanına park eder.	park PARK tabelası işlendiğinde, görevin son adımı.

Sabitler

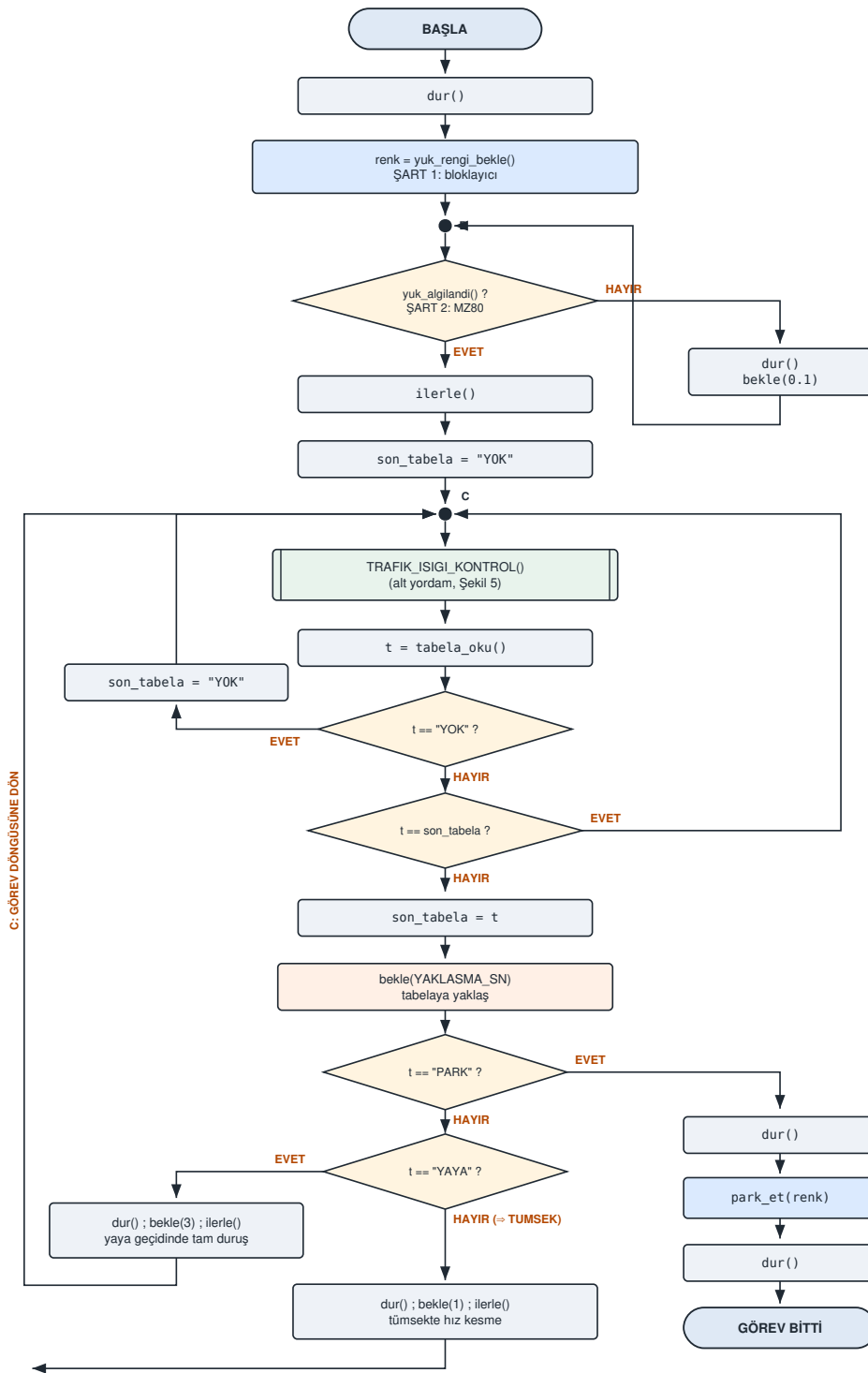
Sabit	Değer	Neden bu değer
YAKLASMA_SN	1,5 s	Tabela kamerada belli bir mesafeden görünüyor. Sabit süre ilerlemek görevi her seferinde aynı noktada uygulamamızı sağlıyor. Saha denemesiyle ayarlanacak.
YAYA_BEKLEME_SN	3,0 s	Yaya geçidindeki tam duruşun dışarıdan net görülebilmesi için.
TUMSEK_BEKLEME_SN	1,0 s	Hız kesmenin eldeki eylemlerle en yakın karşılığı. Varsayım V-3'e bakınız.
RENK_DENEME_MAX	5	Renk okumada tekrar üst sınırı, sonsuz döngüye girmemek için.
YOKLAMA_SN	0,1 s	Beklerken işlemciyi meşgul etmemek için. 100 ms'lik tepki gecikmesi bu hat hızında fark etmiyor.

3. Robot kol akış şeması



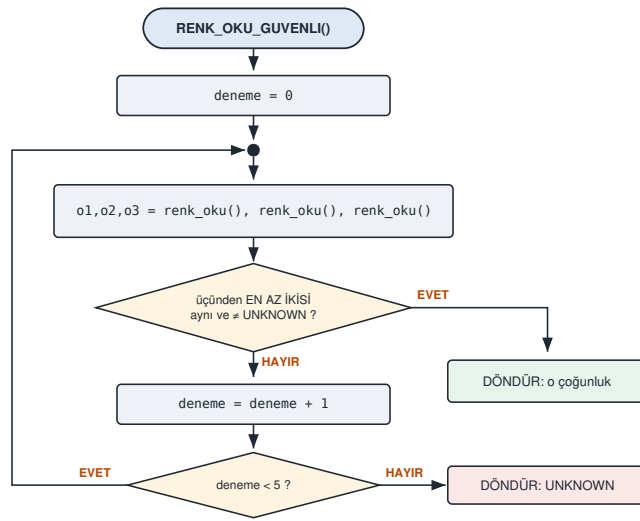
Şekil 2: Robot kol akış şeması

4. Otonom araç akış şeması

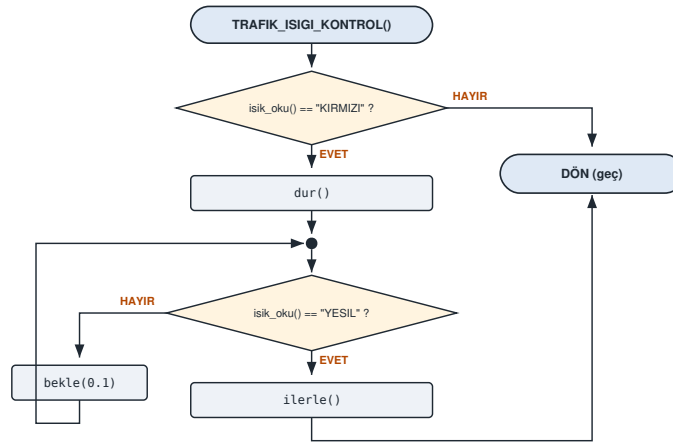


Şekil 3: Otonom araç akış şeması

5. Alt yordamlar



Şekil 4: Alt yordam: RENK_OKU_GUVENLI ()



Şekil 5: Alt yordam: TRAFIK_ISIGI_KONTROL ()

6. Sözde kod

Aşağıdaki sözde kod Şekil 2 ve Şekil 3'teki akış şemalarının birebir karşılığı. İki süreç fiziksel olarak ayrı denetleyicilerde eşzamanlı koşuyor: kol tarafında Arduino ile Jetson, araç tarafında Jetson ile ESP32. Aralarındaki tek bağ MQTT mesajı.

Sabitler

SABITLER

```
YAKLASMA_SN      <- 1.5  # tabela goruldukten sonra yaklasma suresi
YAYA_BEKLEME_SN  <- 3.0  # yaya gecidinde tam durus
TUMSEK_BEKLEME_SN <- 1.0  # tumsekte hiz kesme
RENK_DENEME_MAX   <- 5
YOKLAMA_SN        <- 0.1
GECERLI_RENKLER   <- { "RED", "GREEN", "BLUE" }
```

Süreç A: robot kol, sonsuz döngü

YORDAM ROBOT_KOL_ANA()

```
# ---- Baslangic. Dongunun disinda, yalnızca bir kez -----
home()                # konum bilinmiyor, referansa git
birak()               # tutucuda kup kalmis olabilir
git("GORME")          # kameranin konveyoru gordugu bekleme pozu
```

```
DONGU SONSUZ                                                # (A)
```

```
# ---- 1) PLC'nin basla sinyalinini bekle -----
# Sinyal, kup konveyor cikisindayken VE arac yukleme
# noktasinda dururken geliyor. Bu sarti PLC garanti ediyor,
# kol ayrica kontrol etmiyor.
WHILE NOT basla_geldi()
    bekle(YOKLAMA_SN)
END WHILE
```

```
# ---- 2) Rengi guvenli bicimde oku -----
renk <- RENK_OKU_GUVENLI()
```

```
EGER renk GECERLI_RENKLER icinde DEGILSE
    # Yanlis renk yanlis parkalani demek. Tahmin etmek
    # yerine cevrimi kesiyoruz.
    HATA_BILDIR("renk belirlenemedi")
    # kup ALINMAZ, araca hicbir sey GONDERILMEZ
```

```
DEGILSE
    # ---- 3) Kupu al ve araca yukle -----
    git("AL")
    tut()
    git("YUKLE")
    birak()
```

```
# ---- 4) Kolu guvenli poza cek, ANCAK SONRA bildir --
# Sira kritik: arac, renk bildirimini alir almaz
# hareket edebiliyor.
git("GORME")
araca_gonder(renk)
END EGER

# ---- 5) Ayni sinyalle ikinci cevrimi onle -----
# PLC sinyali SEVIYE olarak suruyor. Dusmesini beklemezsek
# bos bir cevrim daha baslatiyoruz.
WHILE basla_geldi()
    bekle(0.05)
END WHILE
```

```
END DONGU                                     # -> (A) dongu basi
```

```
END YORDAM
```

Alt yordam: RENK_OKU_GUVENLI

```
ALT YORDAM RENK_OKU_GUVENLI() -> renk
deneme <- 0
WHILE deneme < RENK_DENEME_MAX
    o1 <- renk_oku()
    o2 <- renk_oku()
    o3 <- renk_oku()
    c <- COGUNLUK(o1, o2, o3)    # ucunden en az ikisinde ayni
                                # olan deger, yoksa YOK doner
    EGER c, YOK ve "UNKNOWN" degilse ISE
        DONDUR c
    END EGER
    deneme <- deneme + 1
END WHILE
DONDUR "UNKNOWN"                # 5 denemede de kararsiz, hata cagrisi
END ALT YORDAM
```


Süreç B: otonom araç, tek görevlik

YORDAM OTONOM_ARAC_ANA()

```

dur()                                # baslangicta kesinlikle hareketsiz

# ---- Birinci sart: MQTT'den renk bildirimi, bloklayici -----
renk <- yuk_rengi_bekle()

# ---- Ikinci sart: MZ80 kupu fiziksel olarak algilamali -----
WHILE NOT yuk_algilandi()
    dur()
    bekle(YOKLAMA_SN)
END WHILE

# Buraya ancak IKI SART DA saglandiginda geliniyor.
ilerle()
son_tabela <- "YOK"

DONGU SONSUZ                          # (C)

# ---- Trafik isigi her turda kontrol ediliyor -----
TRAFIK_ISIGI_KONTROL()

t <- tabela_oku()

# ---- Kilit acma: tabela gorus alanindan cikti -----
EGER t = "YOK" ISE
    son_tabela <- "YOK"
    DEVAM
END EGER

# ---- Kilit: ayni tabela ikinci kez islenmiyor -----
# Bir tabela kamerada onlarca kare gorunuyor. Kilit
# olmadan arac ayni gorevi defalarca uyguluyor.
EGER t = son_tabela ISE
    DEVAM
END EGER

son_tabela <- t                        # kilitle
bekle(YAKLASMA_SN)                    # tabela uzaktan gorunur, yaklas

SEC t
    DURUM "PARK":
        dur()
        park_et(renk)                # renk birinci sarttan geldi
        dur()
        CIK DONGU                    # arac TEK GOREVLIKTIR

    DURUM "YAYA":
        dur()
        bekle(YAYA_BEKLEME_SN)      # tam durus

```

```
        ilerle()

        DURUM "TUMSEK":
            dur()
            bekle(TUMSEK_BEKLEME_SN)    # hiz kesme, bkz. V-3
            ilerle()
        END SEC

    END DONGU                                # -> (C) gorev dongusu

    # Gorev tamamlandi, surec burada sona eriyor.

END YORDAM
```

Alt yordam: TRAFIK_ISIGI_KONTROL

```
ALT YORDAM TRAFIK_ISIGI_KONTROL()

    EGER isik_oku() "KIRMIZI" DEGILSE
        DONDUR                # isik yok veya yesil, ilerlemeye devam
    END EGER

    # Kirmizi gorundu.
    dur()
    WHILE isik_oku() "YESIL" DEGIL    # CIKIS SARTI YALNIZCA "YESIL"
        bekle(YOKLAMA_SN)            # "YOK" okumak GECME IZNI DEGIL
    END WHILE
    ilerle()

END ALT YORDAM
```

7. Tasarım kararları

Görev tanımında cevaplanması istenen on iki karar noktası var. Aşağıda her birini, verdiğimiz kararlarla birlikte neden başka türlü yapmadığımızı da anlatarak yanıtladık.

Robot kol

Sistem ilk açıldığında, döngüye girmeden önce yapılması gereken bir şey var mı?

Var, üç adım: `home()`, sonra `birak()`, sonra `git("GORME")`. Üçü de döngünün dışında ve yalnızca bir kez çalışıyor.

`home()` gerekiyor çünkü güç kesintisi ya da acil duruş sonrası kol herhangi bir pozda kalmış olabilir. Öğretilmiş pozlara güvenli bir yörünge ancak bilinen bir referans noktasından üretilebiliyor. Bu adımı atlarsak ilk hareket öngörülemez bir yoldan geçiyor ve konveyöre ya da araca çarpma riski doğuyor.

`birak()` koyduk çünkü önceki çevrim yükleme anında kesilmişse tutucuda küp kalmış olabilir. Tutucu kapalıyken `tut()` çağırmak yeni küpü kavramıyor ve çevrim sessizce boşa dönüyor.

`git("GORME")` ise renk okumasının bu pozda yapılacak olması yüzünden. Böylece kol sinyali beklerken zaten doğru yerde duruyor. Sinyal geldikten sonra poza gitmek çevrim süresine gereksiz hareket ekliyor, süre de finalde ayrı bir puan kalemi.

Başla sinyalini beklemeyi nasıl kurguladınız?

Yoklamalı bekleme kullandık: `WHILE NOT basla_geldi(): bekle(0.1)`.

Sinyali PLC seviye olarak sürüyor, yani K10 rölesi robot çevrimi boyunca çekili kalıyor. Dolayısıyla beklediğimiz şey sinyalin gelmesi değil, var olması. `bekle(0.1)` ile araya boşluk koymamız sıkı döngünün işlemciyi gereksiz meşgul etmesini önüyor. 100 ms'lik tepki gecikmesi bu hattın çevrim süresi yanında önemsiz.

Beklemenin ikinci yarısı çevrimin sonunda: iş bittikten sonra sinyalin düşmesini bekliyoruz. İkisi birlikte bir kenar yakalama davranışı üretiyor.

Rengi ne zaman ve kaç kez okursunuz? UNKNOWN gelirse ne olur?

Kol GORME pozundayken, yani küp konveyör çıkışında dururken ve kavramadan önce okuyoruz. Kavrandıktan sonra okumanın iki riski var: tutucu küpün bir yüzünü kapatabiliyor, kolun gövdesi de küpün üzerine gölge düşürebiliyor. HSV eşikleri gölgeye duyarlı olduğu için bu iki durum da okumayı bozuyor.

Üç kez okuyup çoğunluk oyu alıyoruz (Şekil 4). Tek okuma o anki parlamadan veya otomatik pozlama dalgalanmasından etkilenebiliyor. Üç okumadan ikisinin aynı çıkması rastgele gürültüyü büyük ölçüde eliyor, maliyeti de birkaç yüz milisaniye.

UNKNOWN gelirse okumayı en fazla beş kez tekrarlıyoruz. Beş denemede de karar verilemezse küpü almıyoruz ve araca hiçbir şey göndermiyoruz, çevrimi hata bildirimiyle kesiyoruz.

Tahmin etmemeyi bilerek seçtik. Renk, aracın hangi park alanına gideceğini belirliyor. Yanlış bir renk göndermek aracı yola çıkarıp görevi hatalı tamamlatıyor. Bu, hiç başlatmamaktan daha kötü, çünkü hatayı ancak parkta fark ediyoruz ve o noktada düzeltmek mümkün olmuyor. Ayrıca UNKNOWN çoğu zaman küpün yanlış konumlandığını gösteriyor, bu da operatör müdahalesi isteyen fiziksel bir sorun.

Alma ve yükleme hangi sırayla yapılır?

`git("AL"), tut(), git("YUKLE"), birak(), git("GORME")`.

Her hareket komutu ile tutucu komutu arasında sıra bağımlılığı var. Kol hedefe varmadan tutucu kapanırsa küp kavranmıyor, hedefe varmadan açılırsa küp yanlış yere düşüyor. Bu yüzden `tut()` yalnızca AL pozuna varıldıktan, `birak()` yalnızca YUKLE pozuna varıldıktan sonra çağrılıyor.

Sondaki `git("GORME")` iki işi birden yapıyor: bir sonraki çevrimin bekleme pozunu hazırlıyor ve kolu aracın

hareket koridorundan çıkarıyor.

Rengi araca ne zaman gönderirsiniz?

Çevrimin en son adımında, küp bırakıldıktan ve kol GORME pozuna çekildikten sonra. Tasarımdaki en kritik sıralama kararı bu.

Araç iki şartla hareket ediyor: MZ80'in küpü algılaması ve renk bildiriminin gelmesi. Küp araca bırakıldığı anda MZ80 zaten aktif oluyor. Rengi o anda gönderirsek ikinci şart da sağlanıyor ve araç hemen hareket ediyor. Oysa kol henüz araç üzerinden çekilmemiş durumda. Sonuç: araç inen kolun altından kayıyor, küp devriliyor veya kol ile araç çarpışıyor.

Bildirimi en sona almak bu çarpışmayı bir zamanlama ayarına değil, algoritmanın yapısına bağlıyor. Bir faydası daha var: renk erken gönderilip kavrama sonradan başarısız olsaydı araç boş yola çıkmış olurdu. Bu sıralama onu da imkânsız kılıyor.

Kol geri döndüğünde sinyal hâlâ aktifse veya değilse ne olur?

Sinyal hâlâ aktifse bu yeni bir küp geldiği anlamına gelmiyor. PLC sinyali seviye olarak sürüyor ve küp banttı kalkıp BP2 sensörü boşalana kadar çekili kalabiliyor. Bu yüzden algoritma önce sinyalin düşmesini bekliyor: `WHILE basla_geldi() : bekle(0.05)`. Bu adım olmasa aynı küp için ikinci bir çevrim başlıyor, kol boş bir kavrama yapıyor ve araca ikinci kez renk gönderiyor.

Sinyal düşmüşse doğrudan döngü başına dönüyoruz ve yeni sinyali bekliyoruz. İkisi birlikte kenar yakalama davranışı üretiyor: düşen kenar, ardından yükselen kenar. Seviye sinyaliyle çalışan her el sıkışmanın standart çözümü bu.

Süreç nasıl tekrarlanır, nerede başlar nereye döner?

Başlangıç bloğu döngünün dışında ve program ömrü boyunca bir kez çalışıyor. Döngü Şekil 2'de A ile işaretli noktadan başlıyor, sinyalin düşmesi beklendikten sonra yine A'ya dönüyor.

Hata dalı da aynı noktaya dönüyor ama doğrudan değil, önce sinyalin düşmesini bekleyerek. Böylece hata sonrasında da aynı küp için sonsuz tekrar oluşmuyor.

Görev tanımı çevrimin konveyöre küp geldikçe sürekli tekrarlanmasını istediği için döngünün çıkışı yok, süreç yalnızca güç kesildiğinde sona eriyor.

Otonom araç

Araç ne zaman harekete başlar? İki şarttan yalnızca biri sağlanırsa ne olur?

Araç yalnızca iki şart birlikte sağlandığında hareket ediyor. Birincisi `yuk_rengi_bekle()` fonksiyonunun MQTT'den bir renk döndürmesi, ikincisi `yuk_algilandi()` değerinin True olması, yani MZ80'in küpü fiziksel olarak görmesi.

Yalnızca renk gelirse küp araca yüklenmemiş demektir. Kavrama başarısız olmuş ya da küp düşmüş olabilir. Araç boş yola çıkarsa görevi tamamlıyor ama yükü taşıyor, bu da görevin amacını ortadan kaldırıyor. O yüzden `dur()` konumunda bekliyor.

Yalnızca MZ80 algılasa küp araçta ama rengini bilmiyoruz. Renk, park alanı seçiminin tek girdisi, onsuz araç nereye gideceğini bilemiyor. Rastgele bir parka gitmek beklemekten kötü olurdu, o yüzden bekliyor.

Algoritmadaki sıra bilinçli: önce bloklayıcı `yuk_rengi_bekle()`, sonra `yuk_algilandi()` yoklaması. Renk zaten kol tarafından en son gönderildiği için, renk geldiğinde küp fiziksel olarak araçta olmuş oluyor. İkinci kontrol bunun doğrulanması.

Kırmızıda beklemeyi nasıl kurguladınız? Yeşile dönüşü nasıl anlıyorsunuz?

Ayrı bir alt yordamda ele aldık (Şekil 5). Mantık iki aşamalı. Okuma KIRMIZI değilse hiçbir şey yapmıyoruz, araç ilerlemeye devam ediyor. Okuma KIRMIZI ise `dur()` çağırıp bekleme döngüsüne giriyoruz. Bu döngüden

çıkış şartı yalnızca YESİL okumak.

Buradaki kritik ayrım şu: YOK geçme izni değil. `isik_oku()` üç değer döndürüyor ve YOK ışığın görülemediğini söylüyor, söndüğünü değil. Kırmızıda beklerken araç titreşimi, güneş yansıması veya anlık bir algılama kaybı YOK üretebiliyor. Çıkış şartını "KIRMIZI değil" diye yazsaydık araç kırmızı ışıktaki geçirdi. Bu kesin olarak yasak, o yüzden çıkış şartını olumlu biçimde YESİL olarak yazdık.

Zaman aşımı bilerek koymadık. Işık arızalanırsa araç süresiz bekliyor. Alternatif, yani belirli bir süre sonra geçmek, kural ihlali riskini yapısal hâle getirirdi. Beklemek kural ihlalinden daha düşük maliyetli bir başarısızlık.

Tabela görüldüğü an mı durulur, yaklaşınca mı? Aynı tabela iki kez görülürse ne olur?

Görüldüğü an durmuyoruz. Tabela kamerada belli bir mesafeden görünüyor, o anda durursak yaya geçidinin metrelerce önünde durmuş oluyoruz ve görev doğru yerde uygulanmıyor. Bunun yerine tabela algılandığında araç YAKLASMA_SN kadar ilerlemeye devam ediyor, görevi sonra uyguluyor. Kameranın görüş mesafesi ve aracın hızı sabit olduğu için bu süre tekrarlanabilir bir konum veriyor, saha denemesiyle ayarlanacak.

Aynı tabela iki kez görülürse diye kilit koyduk. Bir tabela, araç ona yaklaşırken onlarca ardışık karede görünüyor. Kilit olmasa araç aynı yaya geçidinde defalarca durup kalkardı. Çözüm `son_tabela` değişkeni: bir tabela işlendiğinde türü buraya yazılıyor, aynı tür tekrar okunduğu sürece hiçbir şey yapılmıyor, `tabela_oku()` YOK döndürdüğünde yani tabela görüş alanından çıktığında kilit açılıyor.

Bu yaklaşım pistte aynı türden ikinci bir tabela bulunmasını da doğru işliyor. Araç birinci tabelayı geçiyor, arada YOK okuyor, kilit açılıyor ve ikinci tabela normal şekilde işleniyor.

Park yeri neye göre seçilir? Renk bilgisi hiç gelmezse araç ne yapar?

Park yeri tek bir girdiye göre seçiliyor: sürecin başında `yuk_rengi_bekle()` ile aldığımız renk. PARK tabelası işlendiğinde `park_et(renk)` çağrılıyor. Rengi bir kez alıp değiştirmede tuttuğumuz için park anında yeniden okumaya veya MQTT'ye ihtiyaç yok. Bu sayede park sırasında olası bir ağ kesintisi görevi etkilemiyor.

Renk hiç gelmezse araç zaten hiç hareket etmemiş oluyor. `yuk_rengi_bekle()` bloklayıcı, süreç orada duruyor ve `ilerle()` çağrısına asla ulaşılmıyor. Yani "araç yolda ama rengi bilmiyor" durumu bu tasarımda oluşmıyor. Sorunu çalışma zamanında çözmek yerine akış yapısıyla ortadan kaldırmış oluyoruz.

Park tamamlanınca ne olur?

`park_et(renk)` döndükten sonra `dur()` çağırıp görev döngüsünden çıkıyoruz. Araç süreci sona eriyor, yeni bir görev beklemiyor. Görev tanımı da aracın tek görevlik olduğunu ve park ile birlikte duracağını söylüyor. Sondaki `dur()`, `park_et()` içindeki son manevradan sonra artık hiçbir hareket komutunun kalmadığını garanti ediyor.

Burada iki sürecin ömrü ayrışıyor. Robot kol döngüsü devam ediyor ve konveyöre gelen sonraki küp için yeniden başla sinyali bekliyor. Sistem bir bütün olarak çalışmaya devam ediyor, duran yalnızca o aracın görevi.

8. Güvenlik kurallarına uyum

Görev tanımının sonunda üç kesin kural var. Aşağıdaki tablo her kuralı algoritmada nerede ve nasıl karşıladığımızı gösteriyor.

Kural	Karşılandığı yer	Nasıl
Araç yerinde değilken yükleme yapılmamalı, başla sinyali bunu garanti ediyor.	Robot kol, adım 1, <code>basla_geldi()</code>	Kol araç varlığını kendisi sorgulamıyor. PLC, BP2 (küp) ve BP4 (araç) sensörlerinin ikisini birden sağladığında sinyali veriyor. Kol yalnızca bu sinyale bakıyor. Tek doğrulama noktası olduğu için çift kontrolden kaynaklanan tutarsızlık riski de yok.
MZ80 algılaması ve renk bildirimi birlikte sağlanmadan araç hareket etmemeli.	Otonom araç, birinci ve ikinci şart	<code>ilerle()</code> çağrısına hem <code>yuk_rengi_bekle()</code> döndükten hem de <code>yuk_algilandi()</code> True olduktan sonra ulaşıyor. Tek şartla ilerleyen hiçbir yol yok. Bu bir zamanlama tercihi değil, akışın kendi yapısı.
Kırmızı ışıktaki kesinlikle geçilmemeli.	TRAFIK_ISIGI_KONTROL	Bekleme döngüsünden çıkış şartı yalnızca YESIL okumak. YOK okumak beklemeyi sürdürüyor. Zaman aşımıyla geçiş yolunu bilerek koymadık.

9. Varsayımlar

Aşağıdakiler görev tanımında doğrudan belirtilmeyen ama tasarımın dayandığı noktalar. Her birinin yanında, aksi ortaya çıkarsa algoritmanın hangi tek noktasında değişiklik gerekeceğini de yazdık.

No	Varsayım	Aksi hâlde değişecek yer
V-1	<code>tabela_oku()</code> dönüş kümesi YAYA, TUMSEK, PARK ve YOK olarak verilmiş, HEMZEMIN bu kümede yok. Görev 5 kapsamında hemzemin sınıfını da eğittik ama Görev 1'in hazır eylemi onu döndürmediği için akışta hemzemin dalı yok.	Finalde hemzemin bir görev olarak eklenirse Şekil 3'teki tabela dallanmasına tek bir DURUM "HEMZEMIN" satırı ekleniyor. Kilitleme ve yaklaşma mantığı aynen geçerli kalıyor.
V-2	<code>bekle(sn)</code> bloklayıcı ve bekleme sırasında araç son verilen hareket komutunu koruyor. Yani <code>ilerle()</code> sonrası <code>bekle()</code> çağrısı boyunca araç ilerlemeye devam ediyor.	Bu yüzden yaya ve tümsek dallarında görev önce <code>dur()</code> ile başlıyor. Aksi davranışta, yani bekleme sırasında otomatik duruş olursa, yalnızca YAKLASMA_SN yorumu değişiyor.
V-3	Hazır eylemler arasında hız ayarı yok, yalnızca <code>ilerle()</code> ve <code>dur()</code> var. Tümsek tabelasının beklenen davranışı olan hız kesmeyi, eldeki eylemlerle en yakın karşılık olan kısa duraklamayla modelledik.	Finalde bir hız komutu verilirse <code>dur()</code> , <code>bekle(1)</code> , <code>ilerle()</code> üçlüsü tek bir yavaş ilerleme çağrısıyla değiştiriliyor.
V-4	<code>basla_geldi()</code> seviye sinyali. PLC'nin K10 kontağı robot çevrimi boyunca kapalı kalıyor ve küp banttı kalkınca açılıyor.	Sinyal darbe olsaydı çevrim sonundaki bekleme döngüsü gereksiz kalırdı. Yine de zararsız, çünkü darbe zaten hemen düşüyor.

No	Varsayım	Aksi hâlde değişecek yer
V-5	yuk_rengi_bekle() bloklayıcı ve arac/yuk konusuna gelen ilk mesajı döndürüyor.	Birden fazla mesaj gelirse ilki esas alınıyor. Kol bir çevrimde tek mesaj gönderdiği için bu durum zaten oluşmuyor.
V-6	Şerit takibi aracın alt seviye kontrolünde sürekli çalışıyor, ilerle() "şeridi takip ederek ilerle" anlamında. Şerit takibi bu algoritmanın konusu değil.	Şerit kaybı ayrı bir hata durumu olarak ele alınırsa görev döngüsüne bir kontrol adımı ekleniyor.
V-7	Robot kolun GORME, AL ve YUKLE pozları önceden öğretilmiş ve aralarındaki yörünge çarpışmasız.	Bu üç poz hazır kabul edildiği için ara nokta tanımlamadık.

10. Özet

Bu algoritmayı ayırt eden üç karar var.

Birincisi, renk bildirimini çevrimin en son adımına koymamız. Araç ancak kol güvenli poza çekildikten sonra hareket edebiliyor, yani kol ile araç çarpışmasını bir zamanlama ayarına değil akışın sırasına bağladık.

İkincisi, kırmızı ışıktan çıkış şartını olumlu yazmamız. "KIRMIZI değil" yerine "YESİL" dedik, böylece algılama kaybının kural ihlaline dönüşmesi imkânsız hâle geldi.

Üçüncüsü, tabela işlemeyi kilitli yapmamız. Bir tabela ancak görüş alanından çıkıp yeniden girdiğinde tekrar işleniyor, aynı görev ardışık karelerde defalarca tetiklenmiyor.

Üçünde de aynı yaklaşımı izledik: hata olasılığını çalışma anında kontrol etmeye çalışmak yerine akışın yapısıyla ortadan kaldırmak.